

CITS4401

Software Requirements & Design

Group Project Phase 2

Group Number: Group 39

Name | Student ID | GitHub Username

Kunyu He	24213379	Y0gnut
Sandra Franco Pynadath	24805925	sandrafranco16
Yuxuan Xi	24319908	dylanXi
Dyutiman Shiv Shiv	24112662	Dyutiman-shiv

System Slice: Application Review, Assignment & Revision Subsystem (REMS)

Proposed Architecture: Three-Tier Web Application

We propose a three-tier architecture. It has three parts: a Web Client (SPA), an Application Server, and a Data Tier. This design keeps presentation, business logic, and data storage separate. This separation helps meet the concurrency control requirement (REQ-CON-01) and the access control requirements (REQ-AC-01-03).

Web Client (SPA): The Web Client is a browser-based single-page application. It shows a different dashboard for each user role: PI, Admin Staff, Committee Member, and Chair. It lets users upload documents, view annotations, and compare two document versions side by side (REQ-DOC-03). It also shows application status and notifications. The Web Client sends all requests to the Application Server through a REST API.

Application Server: The Application Server has four modules.

(1) Workflow & Assignment Module: It tracks the state of each application (Submitted → ... → Approved/Rejected). It enforces the self-assign and chair approve pattern. It also flags high-risk applications and uses optimistic locking (REQ-CON-01).

(2) Review & Annotation Module: It lets committee members add annotations to applications. It moves applications flagged for revision into the Admin queue (REQ-REV-01-04).

(3) Communication & Notification Module: It sends formal revision requests to researchers. It also triggers overdue reminders through the Email Gateway (REQ-REV-03, REQ-NOT-01).

(4) Access Control Module: It checks the user's role before every API request (REQ-AC-01-03).

Document Store: The Document Store is a managed object store, such as AWS S3. It holds the binary content of all uploaded documents. The Application Server manages version metadata. Each new upload is automatically marked as the Current Version (REQ-DOC-01-02).

Relational Database: The Relational Database stores user accounts and roles, application metadata, lifecycle state, assignments, annotations, audit trail records, and notification logs. A relational model is used because it supports transactional integrity.

Email Gateway (External): The Email Gateway is an external email service, such as SendGrid. The Notification Module calls it to send revision requests and overdue reminders to researchers.

Key Data Flows

- A researcher uploads a document. The Application Server creates a version record in the database and stores the file in the Document Store.
- A committee member self-assigns to an application. The Application Server records the request. The Chair receives a notification to approve it. Once approved, the application state changes to Under Review.
- A committee member flags an application as Requires Revision and adds annotations. The Application Server saves the annotations to the database. It then places the application in the Admin revision queue.
- An Admin staff member issues a formal revision request. The Application Server logs this action in the audit trail. The Email Gateway sends a notification email to the researcher.
- If a researcher misses the revision deadline, a background job detects the overdue state. The Application Server then sends an automated reminder through the Email Gateway.

Most Constraining NFRs

REQ-CON-01 (Concurrency Control): The current system uses a shared spreadsheet. This causes data loss when two users write at the same time. We use optimistic locking on all state-changing operations. This means a write will fail if the record has changed since the user last read it. No data is silently overwritten.

REQ-AC-01-03 (Access Control): We put all access checks in one place: the Access Control Module. It checks the user's role and assignment status on every API request. Centralising this check means it cannot be skipped by any part of the system.

REQ-US-01 (Performance — 3s side-by-side load): Documents are stored in a dedicated object store. The store sends files directly to the client using pre-signed URLs. This avoids routing large files through the Application Server, which keeps load times within the 3-second limit.

